Final Term Evaluation

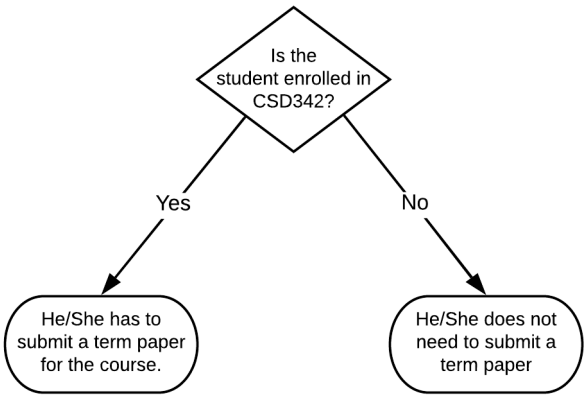
Kabir Kanha Arora
CSD 342 - Data Mining and Data Warehousing
Shiv Nadar University

May 20, 2020

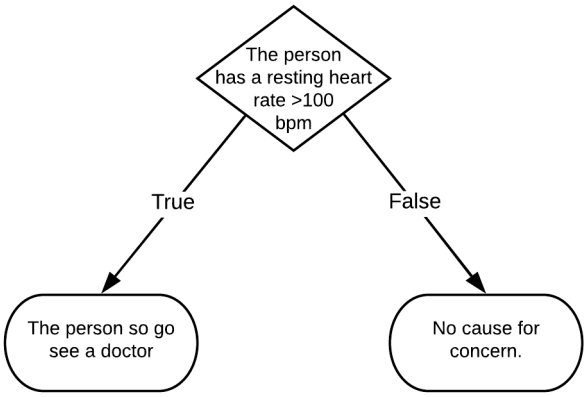
Decision Trees

1 Description

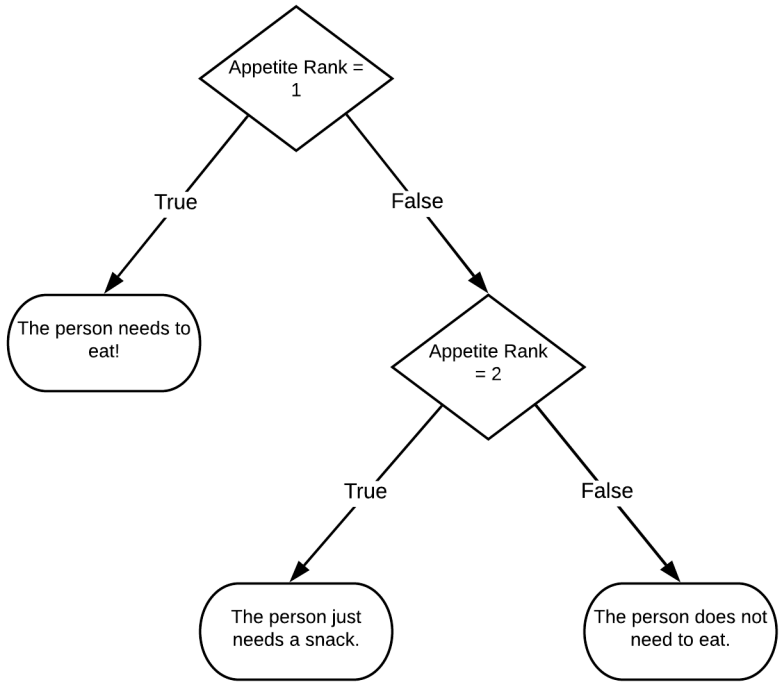
Here is a simple **Decision Tree**:



In general, a decision tree asks a question and then makes a classification based on the answer. The decision tree shown in the example is built on a Yes/No question, but it is just as easy to construct a decision tree based on numeric data. Note that a decision tree classification can result in categories, or even be numeric.



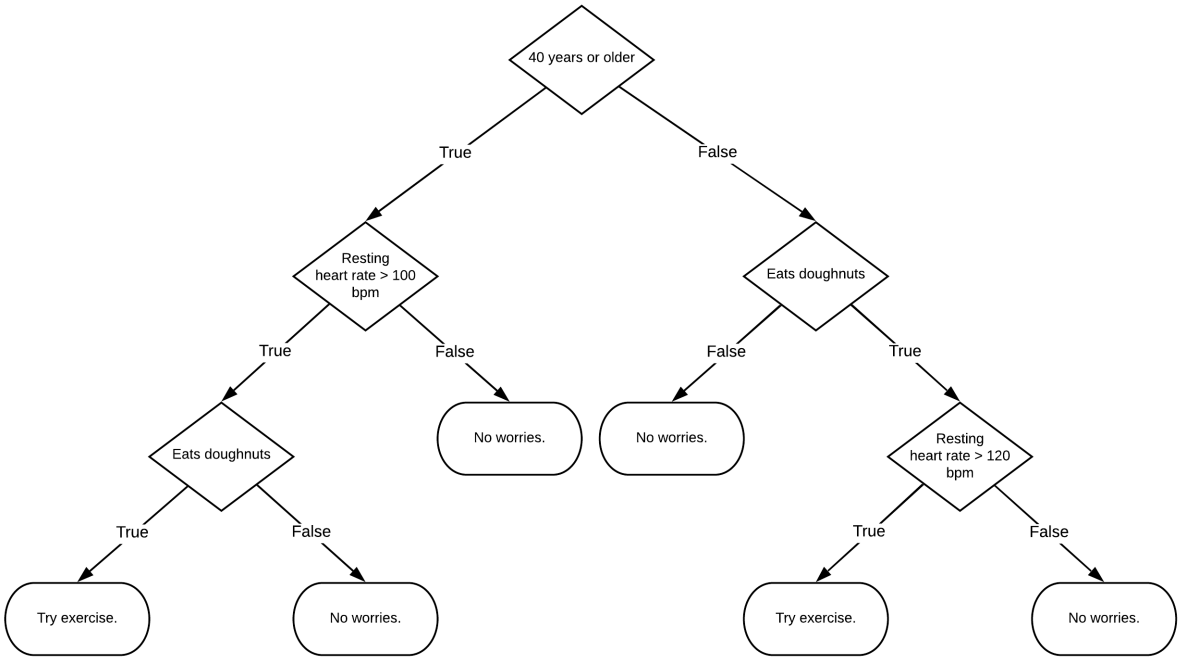
Here is another simple decision tree. This decision tree is based on ranked data, where 1 stands for 'super hungry' and 2 stands for 'moderately hungry'. If the person is super hungry, they need to eat. If the person is moderately hungry, they just need a snack, and if the person is not hungry, they don't need to eat all.



In this term paper, I will try to break everything down into manageable sizes and explain in a way that is understandable to even someone who is reading about decision trees for the first time.

A decision tree is basically a flowchart-like structure with each internal node representing a "question" on an attribute (e.g. if you are a boy or a girl), each branch (arrow) represents the decision of the question, and each leaf node represents a class label (final decision taken after answering all the questions). The paths from root to leaf represent classification rules. A 'target variable' is usually categorical in nature and we use a decision tree to either calculate the probability of each of the categories of the variable, or to classify the variable into the most likely category (or class). Note that decision trees may also be used to estimate the value of a continuous target variable, but it is not suited for that and other models like logistic regression and neural networks usually do a much better job with the same.

Here is a more complicated decision tree, it combines numeric data with Yes/No questions. The questions and their order in different sections of the decision may be different and yet, the classifications may be repeated. Decision trees look pretty intuitive and it's fairly easy to work your way through, from the 'root node', via the 'internal nodes', to the 'leaves'. You start at the top, and work your way down, until you can't go any further.



Note that even though we may have the same nodes or leaves repeated in the tree structure, but it is not allowed for any node to have more than one incoming arrow, This violates the 'tree' condition and is strictly prohibited.

2 Areas of Application

Decision trees, as you may have noticed by now, ask questions and based on that answer, follow a given route. Note that the questions are not always Yes/No and there can be more than two possible decisions at any node. It is evident that the these trees naturally have an "If...Then...Else" structure at every node, and can hence be seamlessly used to model a programmatic structure. They are well suited to categorisation problems where features (or attribute) are systematically checked to make a final classification.
Eg: Decision trees can be used for the COVID-19 self assessment test nowadays by asking simple questions with options and following a different path for every option.
In general, decision trees can be used for the following scenarios:

- Whenever there are several courses of action.
- When the user has an objective he is trying to maximise: such as maximum profit or minimum cost.
- When there is a calculable measure of benefit of the various alternatives.
- When there are events beyond the control of the decision maker.
- When there is uncertainty around which outcome will actually happen.

Some areas where decision trees are being used in real life are:

1. Brand expansion opportunities using historical data.
2. Prediction of the outcome of sports matches using match facts and attributes.
3. determination of the likely buyers of a product.
4. Filtering noise from Hubble space telescope images.
5. Running decision analysis to figure out the best strategy to achieve a goal.
6. Prediction of the likelihood of a loan default using demographic and historic data.

So, we see that decision trees are used in pretty much every industry and sector, ranging from pharmacology to banking to education to law to manufacturing and whatnot.

3 Advantages

Decision trees are one of the simplest classification tool and have several advantages. They are:

1. **Ease of Understanding**: Unlike complicated tools like Support Vector Classifiers, the working of a decision tree is extremely trivial and intuitive to understand, even for a layman. No prior statistical knowledge is required for interpretation and everyone can grasp the model easily once explained.
2. **Easy visualisation**: Decision trees, by the virtue of being a 'tree', are extremely well-suited to graphical representation and this visualisation can help people intuitively understand the situation at hand and they can relate to their hypothesis. Decision trees mirror human decision making more than any other approach.
3. **Useful in data exploration**: Decision trees are one of the fastest ways to identify the most significant variables and the relation between two or more variables. Using decision trees, we can also create new variables (features) that are better suited for predicting the target variable. If we have hundreds of variables, decision trees can help us identify the most significant one(s).
4. **Feature Selection**: Decision trees can also help with variable screening or feature selection. Irrelevant features will be less used so that they can be removed on subsequent runs.
5. **Reduced data preparation**: Decision trees require relatively less effort from users for data preparation. The data cleaning required is also much lesser compared to some other modelling techniques, since they are not influenced much by the presence of outliers and missing values. Some other techniques may need data normalization. Since decision trees can handle qualitative predictors, there is no need to create dummy variables.

6. **Data type not constrained**: Some other modelling techniques are specialised to one type of data, but decision trees can work with both numerical and categorical variables. They can also handle multi-output problems.

7. **Non-Parametric Method**: Decision trees are considered to be a non-parametric method. This means they make no assumptions about the space distribution and classification structure. Also, non-linear relationships between parameters do not affect performance and they are robust against co-linearity, especially boosting.

8. **No Hyperparameter Tuning**: Another reason that makes decision trees easier to use is that unlike some other classifiers like Support Vector Classifier, Decision Trees require virtually no hyperparameter tuning and are always good-to-go as is.

9. **Works with Large Datasets**: A large dataset can be analysed with regular computing resources in a reasonably fast time.

10. **White Box Model**: Unlike Artificial Neural Networks (ANN) which uses a black box model, it is extremely easy to check the working of a decision tree using simple logic-based decisions.

11. **Validation Possible**: The model can easily be validated using statistical tests. This makes it possible to account for the reliability of the model. The model also makes no assumptions about distribution, independence or constant variance.

4 Disadvantages

Had there been no disadvantages, an easy to understand an intuitive model like it would've established a monopoly. The reason that is not the case is because, well, like all other models, decision trees also suffer from certain disadvantages and drawbacks. Some of them are:

1. **Over-fitting**: Decision trees may end up creating havily branched and complex trees that are very well suited to the training data set but do not generalise well, and this is the biggest problem faced by decision tree classifiers. It can usually reduced by pruning and setting constraints on model parameters.

2. **Stability**: Decision trees can be very non-robust. A small change in the training data set can heavily modify the tree constructed and hence, the prediction on the test data set vary wildly. This variance can be lowered using methods like bagging and boosting.

3. **Non-optimality**: Greedy algorithms do not guarantee returning the globally optimal tree. In fact, generating an optimal decision tree is know to be an NP-complete problem. So to reduce the greedy effect of local optimality, newer methods like Dual Information Distance (DID) tree were proposed. We can also mitigate this problem by training multiple trees, where features and samples are randomly sampled with replacement.

4. **Bias**: If some classes dominate, i.e., if the data set is unbalanced, the decision tree created can be biased. To mitigate this, it is always best to balance the data set before fitting to a decision tree.

5. **Accuracy**: Being a straightforward and simple model, decision trees usually exhibit a lower accuracy as compared to other machine learning algorithms.

6. **Information Gain**: When data has categorical variables with different number of labels, the information gain is biased towards attributes with a larger number of labels. This, however, may be avoided by a conditional inference approach, a two-stage approach, or adaptive leave-one-out feature selection.

7. **Continuous Variables**: The decision tree can lose information when working with continuous numerical variables, when it categorises variables in different categories.

8. **Complexity**: The calculations can become very complex when there are many class labels.

5 Algorithm and Improvements

Let us discuss how a decision tree is formed. Let us take an example, where we look at 'Chest Pain', 'Good Blood Circulation' and 'Blocked Arteries' to predict whether a patient has heart disease or not. Assume that we have a data set with these three columns and a fourth with the heart disease status of the patient. The first step would be to figure out which feature comes at the top (root).

We start by looking at how well chest pain alone predicts heart disease, and repeat the exact same thing for the other features. In case some values are missing, we can handle them separately using some techniques like using the most probable entry or use a column with high correlation to is and use it as a guide, or just skip them. For each of the features, we look at the separation data. It is likely that none of the leaf nodes are 100% correct, and hence they all have some degree of 'impurity'. To determine which separation is best, we need some way to measure and compare impurity. There are a bunch of ways to do the same, but we could look at one very popular one called **Gini**.

$$Gini = 1 - \sum_{i=1}^{n} (p_i)^2$$

We calculate the Gini impurity for both nodes for all three scenarios. The total Gini impurity for one scenario is the weighted average of both the nodes. A lower impurity value is desirable. Let us assume that 'Good blood circulation' gives us the lowest impurity. It is hence used as the leaf node. Now we need to figure out how well 'Chest Pain' and 'Blocked arteries' separate the patients on all the current leaf nodes. Note that different features can result in the best separation at different nodes. Also note that this is the property of decision trees

that often leads to non-generalisation and overfitting.At some node, is is also possible that none of the remaining features reduce impurity. In such a case, we simply make that node a leaf node and label it with the majority class. Here each node can also have a probability associated with it. In case a feature never gives us a reduction in impurity, it is dropped. This is a type of an automatic feature selection. However, we can also set some minimum threshold for impurity reduction to facilitate manual feature selection to avoid overfitting.

But what is we had numeric data? How would we determine the best way to divide the patients? Consider the example where we try to classify the patient's heart disease status on the basis of his body weight. Here, what we do is:

1. Sort the patients in ascending order of weight.
2. Calculate the average weight for all adjacent patients.
3. Calculate the impurity value for each adjacent weight.

We will find a given weight value which has the lowest impurity value. Then we make it a node and keep repeating this process till all leaf nodes are finalised.

There are also other algorithms that exist for decision tree learning. Let us look at some of them. The core algorithm for decision trees is called **ID3**. It stands for Iterative Dichotomiser 3 and uses *Entropy* and *Information Gain*,

$$Entropy, H = \sum_{i=1}^{n} -p_i \log_2 p_i$$
$$Gain(T, a) = H(T) - H(T|a)$$

The **Chi-Square** (χ^2) algorithm finds out the statistical significance between the difference between parents and children. A threshold is set to determine whether the distribution of values in the child nodes is different from that in its parent.

$$\chi^2 = \frac{(Actual - Expected)^2}{Expected}$$

Reduction in variance is an algorithm used for continuous target variables (regression problems). Here, we choose the lowest variance.

$$Variance = \frac{\sum(X - \overline{X})^2}{n}$$

The **C4.5 Algorithm** is an improvement on the ID3 algorithm. At each node, C4.5 chooses chooses the feature that results in the best split on the basis of Information Gain. There are a few base cases that this algorithm has. If all the samples in the list at a node belong to

the same class or no information gain is provided by any of the remaining features, then it just creates a leaf at that point. C4.5 makes a range of improvements to ID3. Some of them are:

- Handling both continuous and discrete data - C4.5 sets a threshold and splits the list into 'lower than the threshold' and 'higher than the threshold'.
- Handling training data with missing values - Missing attributes may be marked as '?' and simply not looked at while calculating gain and entropy.
- Pruning trees after creation - C4.5 tried to remove unhelpful and irrelevant branches and replaces them with leaves.

Pruning is required because a tree that is extremely complex and large is prone to overfitting and generalises poorly. But a small tree may not make good enough predictions, it is essential to strike the right balance. It is difficult to know when to stop because it is an NP-complete problem to build an optimal tree. So what is usually done is to construct the entire tree, and then prune the unnecessary and irrelevant branches. Pruning should be done in such a manner that the cross-validated accuracy is not compromised.
Pruning can be of various types, some of them are:

- **Reduced error pruning**: Going from bottom to top, each node is replaced with its most popular class as long as the predictive accuracy is unchanged. This is a simple approach and is fast as well.
- **Cost complexity pruning**: In this method, a series of trees $T_0, T_1, ...T_n$ is created where T_0 is the same as the initial tree and T_n is simply the root node. This is done iteratively, and at step i, T_i is created by removing a sub-tree from T_{i-1} and replacing it with a leaf. The sub-tree to be removed is chosen greedily as the one which minimises what is called the 'error rate'. Once this set of trees is generated, the one with the best performance is chosen.

Pruning has a complementary technique know as **grafting**, which involves the addition of new branches to a leaf.

Boosting is another algorithm aimed at improving the performance. It converts weak learners into strong ones. Weak learners, as the name suggests, are classification models that have a poor performance. Boosting algorithms are of mainly three types:

1. **AdaBoost**: AdaBoost stands for *Adaptive Boosting*. The weak learners in AdaBoost are Small decision tree stumps, i.e., decision trees with just one node split. Initially, all observations are weighted equally, but eventually incorrectly classified observations are given more weight to correct the previous error.
2. **Gradient Bosting**: Gradient Boosting is similar to AdaBoost, but the difference is that here we try to fit the new predictor to the residual errors made in the previous iteration. It is called Gradient Boosting because it uses the *gradient descent* algorithm.

3. **XGBoost**: XGBoost stands for *eXtreme Gradient Boosting* and is a modification of Gradient Boosting designed for performance and speed, which it does by adding the following functionalities to a very slow and unscalable Gradient Boosting algorithm:

    - Parallelisation
    - Distributed Computing
    - Out-of-core Computing
    - Cache optimisation

The **Random Forest** classifier, as the name has it, is a collection of trees (decision trees). In a nutshell, the random forest classifier works by getting a prediction from each of the trees and reporting the vote of the majority. It seems logical to think that this will work as well, if not better than any individual decision tree. This is based on the idea that the majority is usually right in terms of classification. It is essential that the trees are not correlated to each other, otherwise an erroneous prediction can be reflected in all the trees. This is ensured by the following methods:

- **Bagging**: Bagging is short for Bootstrap Aggregation. We discussed that decision trees are unstable and non-robust to changes, i.e., a small change to the training data set can change the decision tree completely. So if we sample the training data set, in all likelihood, we will get a set of decision trees that are extremely different from one another, This is the fundamental property that is exploited by decision trees. However, in bagging, instead of taking a subset of the training data set to sample, we take a sample of the same size as the training set, but we sample with replacement, so some values may occur multiple times, while some won't, at all.

- **Feature Randomness**: Usually, in decision trees we consider all remaining features to select at a node as the splitting feature. However, in a random forest, each tree is allowed to pick only from a random subset of the features. This forces added variation between the trees, and hence lower correlation.

There are also some new variations of the traditional decision tree that have not yet been studied in great depth. Some of them are as follows:

1. **Incremental Decision Tree**: This is an online machine learning variant of the traditional decision tree. Most decision trees use the complete data set in one go, but here we are allowed to update the existing tree using only the new instances, without re-processing the past ones. This is useful in the following situations:

    - The entire data set is not available in the beginning.
    - The data is too large to process in one go.
    - The data keeps getting added to over time.

2. **Alternating Decision Tree**: an ADTree generalises decision trees and is connected to boosting. An ADTree has decision nodes in an alternating fashion, each of which contain a predicate condition and leaf nodes that contain a number. Classification is done by following a true path and summing over all the prediction nodes that are traversed.

3. **Logistic Model Tree**: Here, a decision tree is combined with a Logistic Regression Model. It uses cross-validation to find a number of iterations of LogitBoost so as to not overfit the data.

6 Modifications to suit a distributed environment

In order to do a distributed implementation, one approximation is required. Till now, we looked at every feature value as a split candidate, however, it is costly to find these in a distributed setting. Moreover, we normally want these values to be sorted, and this is a tough ask for distributed computing and requires a lot of computing ind communication. Hence, approximate quantiles are used,[11] which is a standard trade-off that is done to gain improvements in performance without significant loss in accuracy. The first implementation is a **MapReduce** implementation where the map operation or the the flat map in spark terminology would take an instance of the training data as an input and would output a list of split label combinations for all the splitting conditions that it satisfies. After the shuffle, during the reduce operation, a split and a list of corresponding labels is taken and a label histogram for the split is generated. So for k features, m splits/features and n instances, the map operation emits $\mathcal{O}(k * m * n)$ values per best split computation at a node.

This leads to a lot of communication overhead and there is still scope for improvement. so in order to do that, what the authors of this research work did was that they took the training data which was spread over multiple partitions for many machines and they created by aggregated partial statistics on each of these partitions. So they start with an empty array, go through an instance and update the split statistics for every split candidate and come up with the partial statistics and then we combine them to come up with the sufficient statistics. So if we have M splits per feature they can do this pinning operation using pioneered binary search in log M steps versus having to perform M split comparisons. Also when we are updating the been histogram statistics we just need to do it once per feature instead of M times so it gives us significant savings in computation. The third optimization that they did was to see whether they can reduce the passes over this input data set so then they constructed a decision tree they first choose a split candidate at the at node one. When we find the best split we take a filtered version of the original data set with satisfies this fitting condition and perform the same step and we we apply an opposite filter to the to the data set on the right hand side so instead of doing this and moving these data structures in and out of memory we could avoid doing that and just work on top of the cached input data set by applying filters on top of them.

References

[1] Deng, Houtao, et al. "Bias of importance measures for multi-valued attributes and solutions." International conference on artificial neural networks. Springer, Berlin, Heidelberg, 2011.

[2] James, Gareth, et al. An introduction to statistical learning. Vol. 112. New York: springer, 2013.

[3] Mehta, Dinesh, et al. "Decision tree approximations of Boolean functions." Theoretical Computer Science 270.1-2 (2002): 609-623.

[4] Starmer, Joshua. "StatQuest: Decision Trees." StatQuest, 22 Jan. 2018, statquest.org/statquest-decision-trees/.

[5] Brid, Rajesh S. "Decision Trees — A Simple Way to Visualize a Decision." *GreyAtom*, Medium, 26 Oct. 2018, medium.com/greyatom/decision-trees-a-simple-way-to-visualize-a-decision-dc506a403aeb.

[6] Jain, Rishabh. "Decision Tree. It Begins Here." Medium, 20 Mar. 2017, medium.com/@rishabhjain_22692/decision-trees-it-begins-here-93ff54ef134.

[7] D'Souza, Jocelyn. "A Quick Guide to Boosting in ML." Medium, 21 Mar. 2018, medium.com/greyatom/a-quick-guide-to-boosting-in-ml-acf7c1585cb5.

[8] Yiu, Tony. "Understanding Random Forest." *Towardsdatascience*, Medium, 12 June 2019, towardsdatascience.com/understanding-random-forest-58381e0602d2.

[9] Utgoff, Paul E. "Incremental induction of decision trees." Machine learning 4.2 (1989): 161-186.

[10] Freund, Yoav, and Llew Mason. "The alternating decision tree learning algorithm." icml. Vol. 99. 1999.

[11] Sparks, Evan, et al. "Scalable Distributed Decision Trees in Spark MLLib." Databricks, Apache Software Foundation, 17 July 2014, databricks.com/session/scalable-distributed-decision-trees-in-spark-mllib.